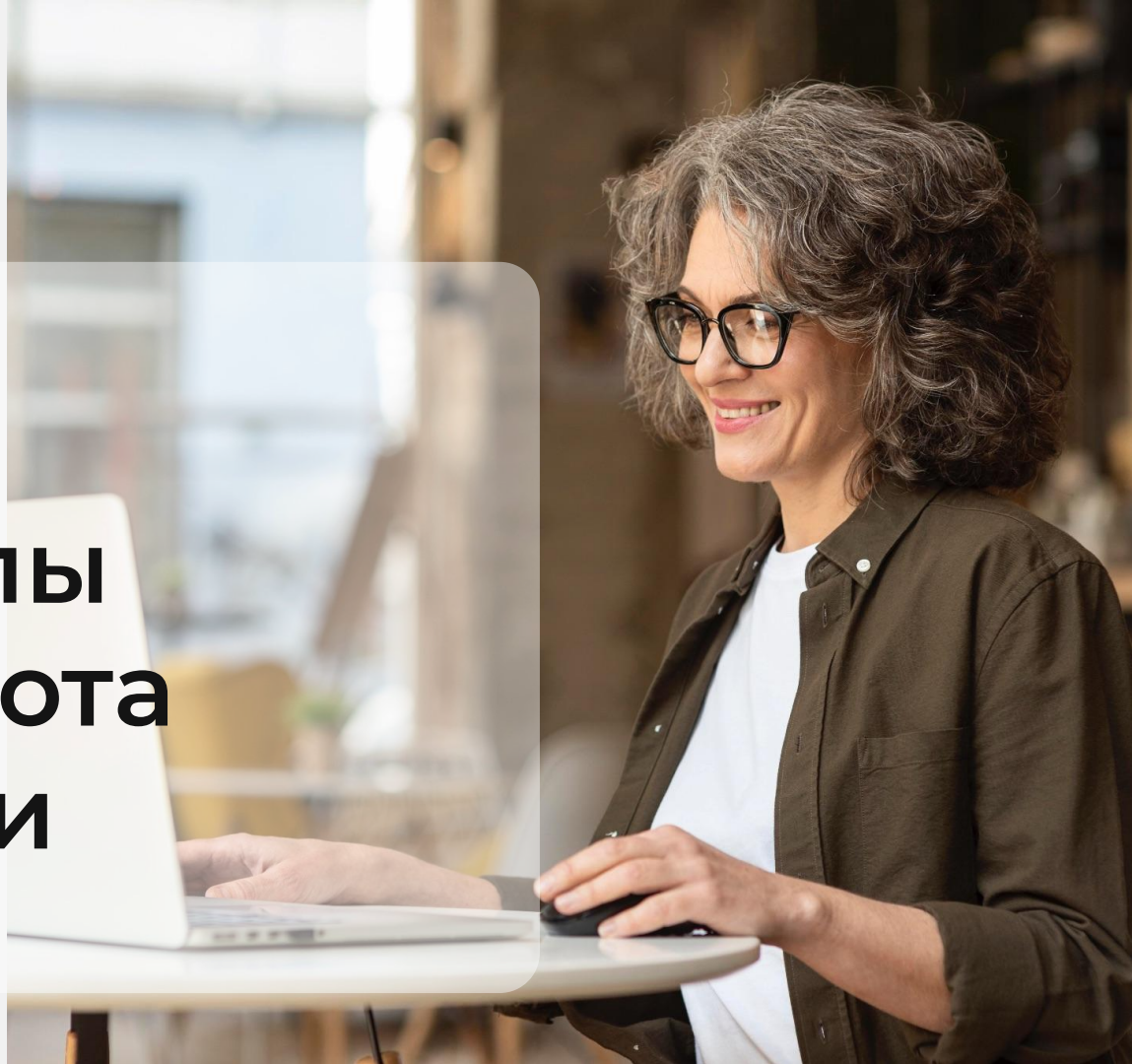


Python

Базовые типы данных, работа с функциями



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

- Особенности языка Python
- Интерпретатор и компилятор
- Jupyter Notebook
- Переменная и оператор присваивания
- Правила именования переменных
- Синтаксис, комментарии и PEP 8
- Функции, вызов функции
- Функция print и функция input

План занятия

- Объекты и типы данных
- Примитивные типы данных
- Возвращаемое значение
- Передача аргументов в функцию
- Функция type
- Строковые операции
- Работа с кавычками в строках
- Экранирование символов
- Параметры sep и end



ОСНОВНОЙ БЛОК





Объекты и типы данных



Объект в Python

Это любое значение, с которым можно работать в программе.

Полезно знать



Объект

Всё в Python — это объект: числа, строки, списки и т.д.

Хранение

Каждый объект хранит данные и имеет определённый тип.



Тип данных

Определяет, какие значения может хранить объект и какие операции можно выполнять с этими значениями.



Динамическая типизация

Python является динамически типизированным языком, что означает, что тип переменной определяется автоматически в момент присваивания значения, и он может меняться в процессе выполнения программы.

Полезно знать



Переменная

Тип данных присваивается не переменной, а объекту, на который она ссылается. Это позволяет одной и той же переменной в ходе программы ссылаться на объекты разных типов данных.

Повторение

`price = 10` # Тип автоматически определен как целое число

`product = "smartphone"` # Тип автоматически переопределен как строка



ВОПРОСЫ





Примитивные типы данных



Примитивные типы данных

Это основные типы, которые используются для хранения простых (единичных) значений.

Примитивные типы данных:



Целые числа (int)



Числа с плавающей запятой (float)



Строки (str)



Логические значения (bool)



Отсутствие значения (NoneType)

Целые числа (int)



Определение

Это значения без дробной части. Они могут быть положительными, отрицательными или равны нулю.

Пример

x = 5 # Положительное целое число
y = -3 # Отрицательное целое число
z = 0 # Ноль

Числа с плавающей запятой (float)



Определение

Это значения с дробной частью.

Пример

```
pi = 3.14          # Число с плавающей запятой  
temperature = -5.2 # Отрицательное число с  
плавающей запятой
```

Экспресс-опрос

?

Чем целое число отличается от числа с плавающей точкой?

Строки (str)



Определение

Это упорядоченные последовательности символов, заключённых в одинарные или двойные кавычки, а также в тройные кавычки (как одинарные, так и двойные).

Пример

```
# Одинарные кавычки
single_quote_string = 'Это строка в одинарных
кавычках.'
```

```
# Двойные кавычки
double_quote_string = "Это строка в двойных
кавычках."
```

Строки (str)



Определение

Тройные кавычки позволяют создавать многострочные строки, сохраняющие исходное форматирование, включая переносы строк.

Пример

```
# Одинарные тройные кавычки (многострочная строка)
triple_quote_string = "Это строка в
тройных кавычках, которая
может занимать несколько строк."
```

Экспресс-опрос



Зачем в строковом типе используется разное число кавычек?

Логические значения (bool)



Определение

Логические значения представляют два возможных состояния: True (истина) и False (ложь). Они записываются текстом с большой буквы без кавычек.

Пример

```
is_student = True      # Логическое значение True
has_passed = False     # Логическое значение
False
```

Ответьте используя булеву логику

?

Идёт ли сейчас урок?

Отсутствие значения (NoneType)



Определение

Тип NoneType представляет отсутствие значения. Единственное значение для этого типа: None

Пример

```
value = None # Переменная, не
              содержит никакого значения
```

Предположите

?

Зачем может применяться отсутствие значения (NoneType)?



Отсутствие значения

Значение None часто используется для инициализации переменных, когда значение ещё не известно, или для обозначения того, что функция ничего не возвращает.



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Какой тип данных будет у переменной x , если присвоить ей значение $x = 5.0$?

- a. str
- b. bool
- c. int
- d. float



Выберите правильный вариант ответа

Какой тип данных будет у переменной x , если присвоить ей значение $x = 5.0$?

- a. str
- b. bool
- c. int
- d. float**



Выберите правильный вариант ответа

Что из перечисленного является логическим значением?

- a. "True"
- b. 0
- c. True
- d. None



Выберите правильный вариант ответа

Что из перечисленного является логическим значением?

- a. "True"
- b. 0
- c. True**
- d. None



Выберите правильный вариант ответа

Что произойдет, если переменной сначала присвоить целое число, а затем строку?

- a. Возникнет ошибка
- b. Тип переменной изменится автоматически
- c. Программа завершится
- d. Значение переменной будет преобразовано в int



Выберите правильный вариант ответа

Что произойдет, если переменной сначала присвоить целое число, а затем строку?

- a. Возникнет ошибка
- b. Тип переменной изменится автоматически**
- c. Программа завершится
- d. Значение переменной будет преобразовано в int



ВОПРОСЫ





**Возвращаемое
значение**



Возвращаемое значение

Это результат, который функция передаёт обратно в место своего вызова.

Возвращаемое значение



Определение

Функции могут возвращать значение, или ничего не возвращать, в последнем случае по умолчанию будет возвращено None.

Результат, который вернула функция, можно присвоить некоторой переменной.

Пример

```
y = print("Привет, мир!")  
x = input()
```

Привет, мир!

x, y

('45', None)



Процедуры в Python

В Python нет отдельного понятия "процедура", как в некоторых других языках программирования.

Процедуры в Python



Определение

Термин процедура часто используется для описания функций, которые ничего не возвращают (или возвращают None) и выполняют определённые действия, такие как вывод на экран или модификация данных.

Пример

```
print("Привет, мир!") # Выводим сообщение, но  
print не возвращает результат
```



ВОПРОСЫ





Передача аргументов в функцию



Передача аргументов в функцию

В функцию можно передавать аргументы
разными способами.

Передача аргументов в функцию



Способ 1

Значения (литералы), например, числа или строки.

Пример

```
print(5)  
print("Привет, мир!")
```

Передача аргументов в функцию



Способ 2

Переменные, из которых будут получены значения.

Пример

```
text = "Привет, мир!"  
print(text)
```

Передача аргументов в функцию



Способ 3

Результат выполнения математических операций.

Пример

```
print(2 + 5)
```

Передача аргументов в функцию



Способ 4

Результаты выполнения других функций.

Пример

```
print(input("Введите имя: "))
```

input возвращает то, что ввел пользователь.

Это значение можно передать в функцию print.

Экспресс-опрос

?

Какой аргумент вернет функция:
`print(2 + 5)?`



ВОПРОСЫ





Функция type



Функция type

Используется для определения типа объекта.
Она возвращает класс типа объекта.

Функция type



Определение

Она возвращает тип данных (класс), который находится в переменной в момент выполнения функции type.

Чтобы увидеть результат вызова функции type, необходимо передать его как аргумент в функцию print.

Пример

```
x = 10
print(type(x)) # <class 'int'>
x = "Привет"
print(type(x)) # <class 'str'>
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Что произойдёт, если передать переменную в функцию `print()`?

- a. Будет выведено имя переменной
- b. Будет выведено значение переменной
- c. Возникнет ошибка
- d. Будет выведен тип переменной



Выберите правильный вариант ответа

Что произойдёт, если передать переменную в функцию `print()`?

- a. Будет выведено имя переменной
- b. Будет выведено значение переменной**
- c. Возникнет ошибка
- d. Будет выведен тип переменной



ВОПРОСЫ





Строковые операции

Строковые операции:



Конкатенация



Умножение строк

Конкатенация



Определение

Конкатенация — это процесс объединения двух или более строк в одну строку. Для этого используется оператор +.

Пример

```
str1 = "Привет"
str2 = "мир"
result = str1 + " " + str2 # Объединение строк с
                             пробелом между ними
print(result) # Вывод: Привет мир
```



Конкатенация

Позволяет создавать новые строки путём добавления одной строки к другой. Это может быть полезно для комбинирования данных или формирования строки для вывода из нескольких элементов.

Умножение строк



Определение

Умножение строк позволяет создать новую строку, состоящую из повторений исходной строки.

Это делается с использованием оператора `*`, за которым следует число, указывающее, сколько раз строка должна быть повторена.

Пример

```
str1 = "Hi! "  
result = str1 * 3 # Создание строки из трёх  
повторений исходной строки  
print(result) # Вывод: Hi! Hi! Hi!
```

Экспресс-опрос

?

В какой ситуации может быть полезно умножение строк?



Умножение строк

Умножение может быть полезно для создания разделителей, повторяющихся шаблонов текста и других целей.

Обратите внимание, что при использовании этих операций изначальные строки не изменяются.



ВОПРОСЫ





Работа с кавычками в строках



Кавычки в Python

В Python строки можно заключать в одинарные ('') или двойные кавычки (""), что позволяет гибко работать с текстом.

Эти два варианта используются взаимозаменяемо, но есть особенности, когда лучше использовать определенные кавычки.

Одинарные и двойные кавычки



Рекомендация

Если внутри строки необходимо использовать кавычки, то целесообразно выбирать противоположные по типу кавычки для оформления строки.

Пример

Одинарные кавычки внутри строки
quote1 = "Она сказала: 'Привет!'"

Двойные кавычки внутри строки
quote2 = 'Он ответил: "Привет!"'

Многострочные строки



Рекомендация

Для многострочных строк и строк, содержащих как одинарные, так и двойные кавычки, удобно использовать тройные кавычки (''' или ''').

В данном случае внутри строки можно использовать как одинарные, так и двойные кавычки.

Пример

```
multi_line_string = """Это многострочная строка, в  
которой есть 'одинарные' и "двойные" кавычки."""
```



ВОПРОСЫ





Экранирование СИМВОЛОВ



Экранирование

Это способ обработки специальных символов внутри строк.

В Python используется обратный слэш (\) для обозначения специальных символов и создания **escape-последовательностей**.



Escape-последовательности

Представляют собой комбинации символов для корректного отображения таких символов, как кавычки, обратный слэш, перенос строки или табуляцию, позволяя включать их в строку без конфликтов с синтаксическими правилами.

Экранирование кавычек



Рекомендация

Если строка обрамлена в одинарные кавычки, а внутри нужно использовать одинарную кавычку, её нужно экранировать. То же правило действует и для двойных кавычек.

Пример

```
# Одинарные кавычки внутри строки
string1 = 'В строке есть \'одинарные\' и "двойные" кавычки.'
print(string1)

# Двойные кавычки внутри строки
string2 = "В строке есть 'одинарные' и \"двойные\" кавычки."
print(string2)
```

Специальные символы



Рекомендация

Экранирование также используется для вставки специальных символов, таких как:

`\n` — новая строка

`\t` — табуляция

Пример

Новая строка

```
text = "Первая строка\nВторая строка"
```

```
print(text)
```

Табуляция

```
text_with_tab = "Первая строка\tВторая строка"
```

```
print(text_with_tab)
```

Экспресс-опрос

?

Для чего используется специальный символ `\t`?

Экранирование обратного слэша



Рекомендация

Если в строке необходимо вставить обратный слэш, его тоже нужно экранировать, так как сам обратный слэш является специальным символом.

Пример

```
path = "C:\\Users\\Username\\Documents"
```



ВОПРОСЫ





Параметры sep и end



Параметры `sep` и `end`

Функция `print` имеет несколько необязательных параметров, которые позволяют настроить ее поведение.

Параметры sep и end



sep (разделитель)

Определяет строку, которая будет вставлена между аргументами. По умолчанию используется пробел.

```
print("one", "two", "three")           # one two three
print("one", "two", "three", sep="--")  #
one--two--three
```

end (конец)

Определяет строку, которая будет добавлена в конец вывода. По умолчанию используется символ новой строки \n.

```
print("Привет", "мир", end="!!")
print("Привет")  # Привет, мир!!Привет
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
str1 = "Привет"
str2 = "мир"
result = str1 + str2
print(result)
```

- a. Привет мир
- b. Приветмир
- c. Привет, мир
- d. Ошибка



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
str1 = "Привет"
str2 = "мир"
result = str1 + str2
print(result)
```

- a. Привет мир
- b. Приветмир**
- c. Привет, мир
- d. Ошибка



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
str1 = "Hi! "  
result = str1 * 3  
print(result)
```

- a. Hi! Hi! Hi!
- b. HiHiHi
- c. Ошибка
- d. Hi!



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
str1 = "Hi! "  
result = 3 * str1  
print(result)
```

- a. **Hi! Hi! Hi!**
- b. HiHiHi
- c. Ошибка
- d. Hi!



Цитата

Создайте программу, которая выведет строку: She said: "It's amazing!" двумя способами (одинаковый результат, но разный код).



Строки

Напишите программу, которая трижды попросит пользователя ввести слово, а затем выведет все три введенных слова одной строкой, через многоточие. Используйте параметр `sep`.



ВОПРОСЫ



Домашнее задание

1. Числовая цепочка

Напишите программу, которая выводит числа от 1 до 5 включительно, разделяя их символами "---". Используйте параметр sep.

2. Пробельные символы

Напишите программу, которая выведет две строки: первая строка с табуляцией, вторая — на новой строке с помощью одного вызова print().

3. Текст в кавычках

Напишите программу, которая выводит строку "Это файл "example.txt"". Выполните задачу несколькими способами.

Заключение

